Correction TD1 CV

import cv2 import numpy as np import random from matplotlib import pyplot as plt

Exercice 1

def exercice1(image_path): # 0. Lire et afficher une image img = cv2.imread(image_path)

```
if img is None:
    print(f"Impossible de charger l'image : {image_path}")
    return

cv2.imshow("Image originale", img)

# 1. Afficher les propriétés
hauteur, largeur, canaux = img.shape
taille = img.size               # nombre de valeurs (pixels * canaux)
profondeur = img.dtype          # type des valeurs (ex: uint8 → 8 bits)

print(f"Image: {image_path}")
print(f" - Définition : {largeur} x {hauteur}")
print(f" - Taille (nb valeurs): {taille}")
print(f" - Nombre de canaux : {canaux}")
print(f" - Profondeur (dtype): {profondeur}\n")

cv2.waitKey(0)
cv2.destroyAllWindows()

# 4. Test mode GRAYSCALE et COLOR
gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
color = cv2.imread(image_path, cv2.IMREAD_COLOR)
unchanged = cv2.imread(image_path, cv2.IMREAD_UNCHANGED)

cv2.imshow("Grayscale", gray)
cv2.imshow("Color", color)
cv2.imshow("Unchanged", unchanged)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Exercice 2

def exercice2(): # 1. Image en niveaux de gris 100x200 avec intensité = 120 img_gray = np.full((100, 200), 120, dtype=np.uint8) cv2.imshow("Image Grise", img_gray)

```
# 2. Image jaune 100x200 (jaune = rouge + vert)
img_jaune = np.zeros((100, 200, 3), dtype=np.uint8)
img_jaune[:, :] = (0, 255, 255)  # OpenCV: BGR → (Bleu, Vert, Rouge)
cv2.imshow("Image Jaune", img_jaune)

# 3. Image aléatoire 200x200
img_random = np.random.randint(0, 256, (200, 200, 3), dtype=np.uint8)
cv2.imshow("Image Aleatoire", img_random)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Exercice 3

def exercice3(image_path): img = cv2.imread(image_path) if img is None: print("Impossible de charger l'image.") return

```
# 1. Séparer les canaux
b, g, r = cv2.split(img)

# Affichage niveaux de gris
cv2.imshow("Canal Bleu - Niveaux de gris", b)
cv2.imshow("Canal Vert - Niveaux de gris", g)
cv2.imshow("Canal Rouge - Niveaux de gris", r)

# Pseudo-images couleur
zeros = np.zeros_like(b)
img_r = cv2.merge([zeros, zeros, r])
img_g = cv2.merge([zeros, g, zeros])
img_b = cv2.merge([b, zeros, zeros])

cv2.imshow("Rouge", img_r)
cv2.imshow("Vert", img_g)
cv2.imshow("Bleu", img_b)

# Vérification recomposition
img_reconstruite = cv2.add(cv2.add(img_r, img_g), img_b)
cv2.imshow("Image Reconstituée", img_reconstruite)

# 2. Affichage côte à côte (utilisation matplotlib)
plt.figure(figsize=(10, 5))
plt.subplot(1, 4, 1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB));
plt.title("Originale")
plt.subplot(1, 4, 2); plt.imshow(r, cmap="gray"); plt.title("Rouge")
plt.subplot(1, 4, 3); plt.imshow(g, cmap="gray"); plt.title("Vert")
plt.subplot(1, 4, 4); plt.imshow(b, cmap="gray"); plt.title("Bleu")
plt.show()

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Exemple d'utilisation

if **name** == "**main**": # Exercice 1 : tester plusieurs images for img_name in ["gateaux1.png", "gateaux2.png", "gateaux3.png", "fleur.png", "lena.png"]: exercice1(img_name)

```
# Exercice 2
exercice2()

# Exercice 3
exercice3("fleur.png")
```